

ACAD 222: Machine Intelligence

Final Project: Part 2

Zeinab Drameh

drameh@usc.edu

Spring 2026

1. Domain

The domain of this project is medical imaging, specifically the classification of chest X-ray images to detect pneumonia. This problem falls within healthcare diagnostics, where machine learning can assist in identifying diseases from imaging data.

Pneumonia is a serious respiratory condition that requires timely diagnosis. Automated classification systems can help support medical professionals by providing faster and more consistent preliminary assessments. This project explores how labeled X-ray images can be used to distinguish between normal and pneumonia cases.

2. Dataset

Source:

Kaggle – Chest X-Ray Images (Normal vs Pneumonia)

Size:

- ~5,800 images total
- Training set: ~5,200 images
- Test set: ~600 images
- Validation set: ~16 images

Structure:

The dataset is organized into folders by class label:

- NORMAL
- PNEUMONIA

Each image represents a chest X-ray labeled according to diagnosis.

Relevance:

This dataset is appropriate because it directly supports a classification task where the goal is to determine whether an X-ray shows signs of pneumonia.

Important Note:

The dataset is not originally in tabular format. To analyze it using pandas, a simple DataFrame was created containing:

- image file paths
- corresponding labels

3. Problem Type

This project is a **classification problem**.

The target variable is categorical:

- NORMAL
- PNEUMONIA

Since the goal is to assign each image to one of these categories, classification is the appropriate approach. In later stages, this problem can be addressed using image-based machine learning models such as convolutional neural networks (CNNs) with pytorch.

4. Attributes

After loading the dataset into a DataFrame, the following attributes were used:

```
INFO:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5856 entries, 0 to 5855
Data columns (total 3 columns):
#   Column      Non-Null Count  Dtype
---  -
0   image_path  5856 non-null   object
1   label       5856 non-null   object
2   label_num   5856 non-null   int64
dtypes: int64(1), object(2)
```

Missing Values

No missing values were found in the dataset after loading the file paths and labels.

Text Attribute

- `image_path` stores the location of each image

- `label` is a categorical variable (NORMAL or PNEUMONIA)

To enable numerical analysis, the label was encoded as:

- NORMAL as 0
- PNEUMONIA as 1

5. Preliminary Correlation Findings

Correlation analysis was performed using:

```
df.corr(numeric_only=True)
```

DESCRIBE:

	label_num
count	5856.000000
mean	0.729679
std	0.444164
min	0.000000
25%	0.000000
50%	1.000000
75%	1.000000
max	1.000000

CORRELATION:

	label_num
label_num	1.0

Observations:

- The dataset contains only one numeric variable (`label_num`), so correlation analysis is limited
- As a result, no meaningful linear relationships can be found using `.corr()`
- This highlights a key limitation of applying traditional statistical methods to image datasets

These findings suggest that simple tabular analysis is insufficient, and that meaningful patterns will need to be extracted directly from the image data in later stages.

Next Steps:

The next steps focus on implementing a PyTorch pipeline to preprocess, load, and model chest X-ray images using CNNs, with the goal of accurately classifying pneumonia cases.

Part 2

ML Algorithms

For this project I am using three algorithms: a Convolutional Neural Network (CNN), a Support Vector Machine (SVM), and a Random Forest classifier. The CNN is the primary algorithm since the input is image data and CNNs are specifically designed to detect spatial patterns in images. The convolutional layers act as automatic feature extractors, learning things like edges, textures, and region-level patterns directly from the raw pixels, which is exactly how pneumonia shows up visually in an X-ray. The SVM and Random Forest are included as comparison baselines. SVMs work well in high-dimensional spaces and are a strong choice when the number of features is large relative to the number of samples. Random Forest is a robust ensemble method that handles noisy data well and provides feature importance scores. Both will be run on flattened pixel vectors extracted from the images and tuned using `RandomizedSearchCV`.

Correlations

Because this dataset is made up of images rather than tabular data, a traditional correlation matrix does not apply. There are no named numeric columns to compare. The equivalent analysis for image classification is comparing pixel intensity distributions across the two classes, which I covered in Part 1. In place of a correlation plot, I am using ROC curves after training to compare how well each of the three algorithms separates the two classes. A higher ROC means the model is better at distinguishing NORMAL from PNEUMONIA. These charts will be added once all three models have been trained.

Transformers

For the CNN, the data transformation pipeline handles all preprocessing. The training set uses random cropping and random horizontal flipping before resizing to 64x64 — this is called data augmentation, and it artificially increases the variety of the training data to reduce overfitting. The validation and test sets only use a center crop and resize with no random augmentation, since we want consistent evaluation. All images are converted to grayscale (1 channel) and normalized to the range $[0, 1]$ via `ToTensor()`. No custom transformer class is needed because `torchvision`'s built-in `transforms.Compose` handles the full pipeline.

For the SVM and Random Forest, the preprocessing is simpler. Since those models take flat numeric vectors as input, the images are resized and flattened into a 1D array of pixel values. A `StandardScaler` is applied to normalize the pixel values before passing them to the models, which is important for SVM performance. This pipeline is handled using `sklearn`'s `Pipeline` class.

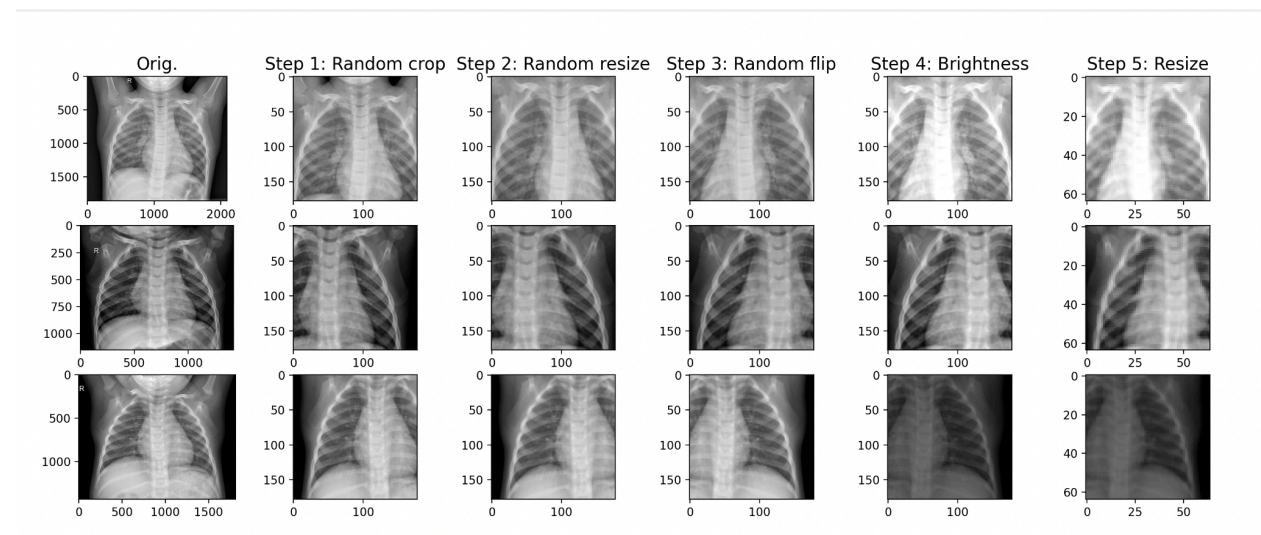
Training

The CNN was trained for 20 epochs using the Adam optimizer and Binary Cross-Entropy loss (`BCELoss`), which is the standard loss function for binary classification with a single output unit.

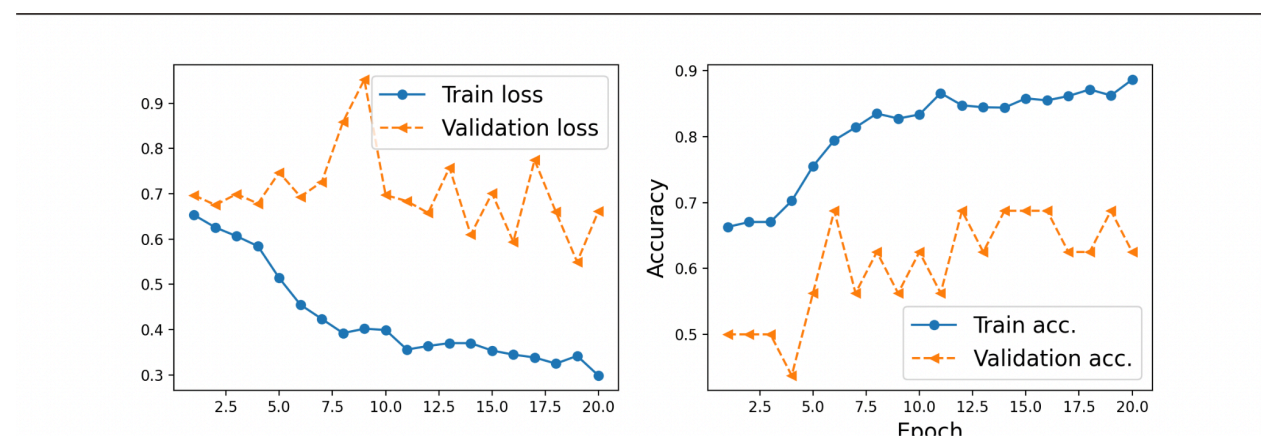
A dropout rate of 0.5 was applied after the first two convolutional layers to reduce overfitting. The model architecture follows four convolutional layers with 32, 64, 128, and 256 feature maps respectively, each using a 3x3 kernel with same padding, followed by max-pooling and a global average-pooling layer to reduce the output down to a single sigmoid unit.

How should the model distinguish between healthy and not healthy lungs? The healthy lung images are clearer and don't present any opacification of the image.

Since there isn't much data to work with, I decided to augment the data in order to create more. I did this by creating a pipeline using 5 random transformations: random crop, random flip, random resize, and random brightness and then a final resizing of the images.

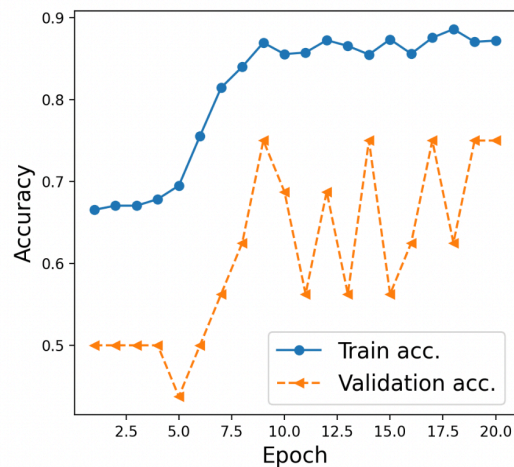
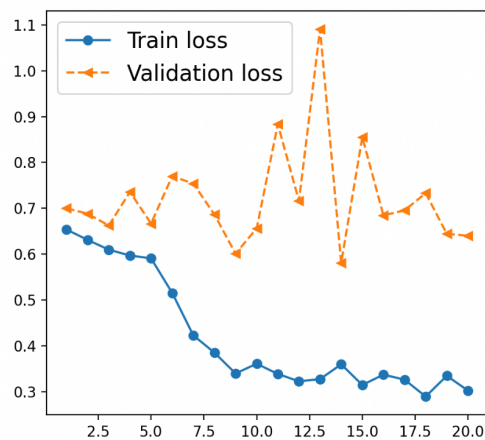


I started off with 20 epochs and a learning rate of 0.001.

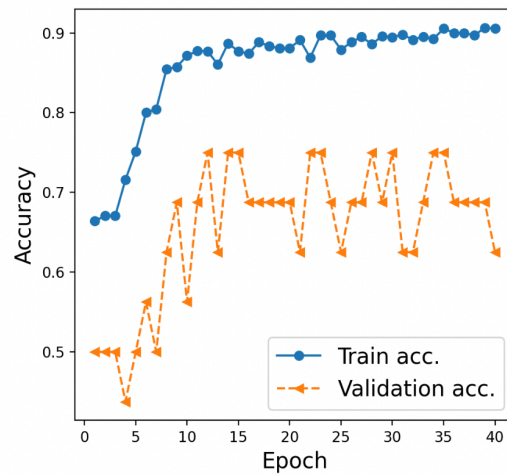
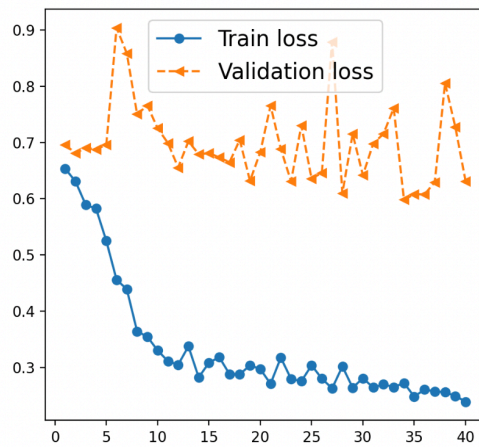


This didn't do very well, the loss curve wasn't great and the model was highly overfitting. I found that brightness is probably not a good parameter to add since it alters the opaque look that distinguishes between the clear and pneumonia lungs. I decided to try again removing the brightness.

```
Validation set: 16
Epoch 1 accuracy: 0.6655 val_accuracy: 0.5000
Epoch 2 accuracy: 0.6705 val_accuracy: 0.5000
Epoch 3 accuracy: 0.6705 val_accuracy: 0.5000
Epoch 4 accuracy: 0.6785 val_accuracy: 0.5000
Epoch 5 accuracy: 0.6950 val_accuracy: 0.4375
Epoch 6 accuracy: 0.7555 val_accuracy: 0.5000
Epoch 7 accuracy: 0.8145 val_accuracy: 0.5625
Epoch 8 accuracy: 0.8400 val_accuracy: 0.6250
Epoch 9 accuracy: 0.8695 val_accuracy: 0.7500
Epoch 10 accuracy: 0.8555 val_accuracy: 0.6875
Epoch 11 accuracy: 0.8575 val_accuracy: 0.5625
Epoch 12 accuracy: 0.8725 val_accuracy: 0.6875
Epoch 13 accuracy: 0.8655 val_accuracy: 0.5625
Epoch 14 accuracy: 0.8550 val_accuracy: 0.7500
Epoch 15 accuracy: 0.8735 val_accuracy: 0.5625
Epoch 16 accuracy: 0.8560 val_accuracy: 0.6250
Epoch 17 accuracy: 0.8755 val_accuracy: 0.7500
Epoch 18 accuracy: 0.8860 val_accuracy: 0.6250
Epoch 19 accuracy: 0.8705 val_accuracy: 0.7500
Epoch 20 accuracy: 0.8720 val_accuracy: 0.7500
```



The model was converging quicker on the training set, but not so much for the validation set. It also seemed to overfit slightly less. It seemed as though the validation set was just beginning to converge quicker so I decided to increase the epochs to see if there would be a difference.



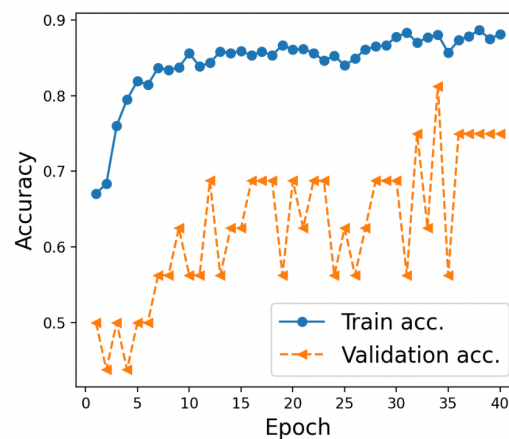
```
Epoch 20 accuracy: 0.8810 val_accuracy: 0.6875
Epoch 21 accuracy: 0.8910 val_accuracy: 0.6250
Epoch 22 accuracy: 0.8690 val_accuracy: 0.7500
Epoch 23 accuracy: 0.8970 val_accuracy: 0.7500
Epoch 24 accuracy: 0.8970 val_accuracy: 0.6875
Epoch 25 accuracy: 0.8790 val_accuracy: 0.6250
Epoch 26 accuracy: 0.8885 val_accuracy: 0.6875
Epoch 27 accuracy: 0.8950 val_accuracy: 0.6875
Epoch 28 accuracy: 0.8860 val_accuracy: 0.7500
Epoch 29 accuracy: 0.8960 val_accuracy: 0.6875
Epoch 30 accuracy: 0.8945 val_accuracy: 0.7500
Epoch 31 accuracy: 0.8980 val_accuracy: 0.6250
Epoch 32 accuracy: 0.8915 val_accuracy: 0.6250
Epoch 33 accuracy: 0.8955 val_accuracy: 0.6875
Epoch 34 accuracy: 0.8925 val_accuracy: 0.7500
Epoch 35 accuracy: 0.9055 val_accuracy: 0.7500
Epoch 36 accuracy: 0.9000 val_accuracy: 0.6875
Epoch 37 accuracy: 0.9000 val_accuracy: 0.6875
Epoch 38 accuracy: 0.8970 val_accuracy: 0.6875
Epoch 39 accuracy: 0.9065 val_accuracy: 0.6875
Epoch 40 accuracy: 0.9055 val_accuracy: 0.6250
```

The training accuracy shot up significantly, while the validation set was getting more consistent yet performing much worse. I decided to change the learning rate to 0.005 to see what differences that might make.

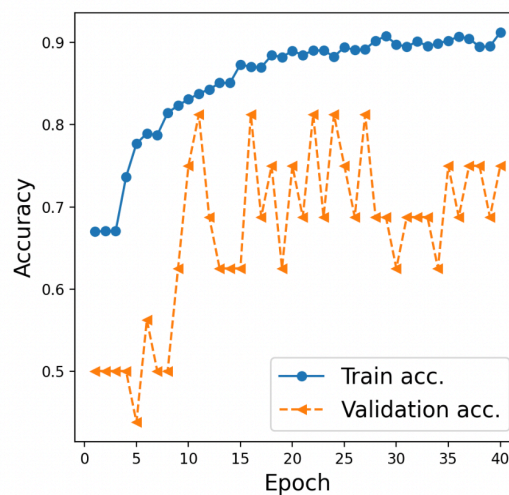
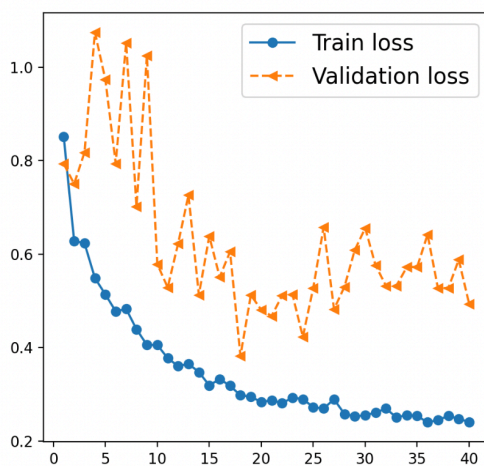

```

Epoch 25 accuracy: 0.8400 val_accuracy: 0.6250
Epoch 26 accuracy: 0.8490 val_accuracy: 0.5625
Epoch 27 accuracy: 0.8615 val_accuracy: 0.6250
Epoch 28 accuracy: 0.8650 val_accuracy: 0.6875
Epoch 29 accuracy: 0.8670 val_accuracy: 0.6875
Epoch 30 accuracy: 0.8780 val_accuracy: 0.6875
Epoch 31 accuracy: 0.8835 val_accuracy: 0.5625
Epoch 32 accuracy: 0.8705 val_accuracy: 0.7500
Epoch 33 accuracy: 0.8770 val_accuracy: 0.6250
Epoch 34 accuracy: 0.8810 val_accuracy: 0.8125
Epoch 35 accuracy: 0.8570 val_accuracy: 0.5625
Epoch 36 accuracy: 0.8735 val_accuracy: 0.7500
Epoch 37 accuracy: 0.8785 val_accuracy: 0.7500
Epoch 38 accuracy: 0.8870 val_accuracy: 0.7500
Epoch 39 accuracy: 0.8750 val_accuracy: 0.7500
Epoch 40 accuracy: 0.8815 val_accuracy: 0.7500

```

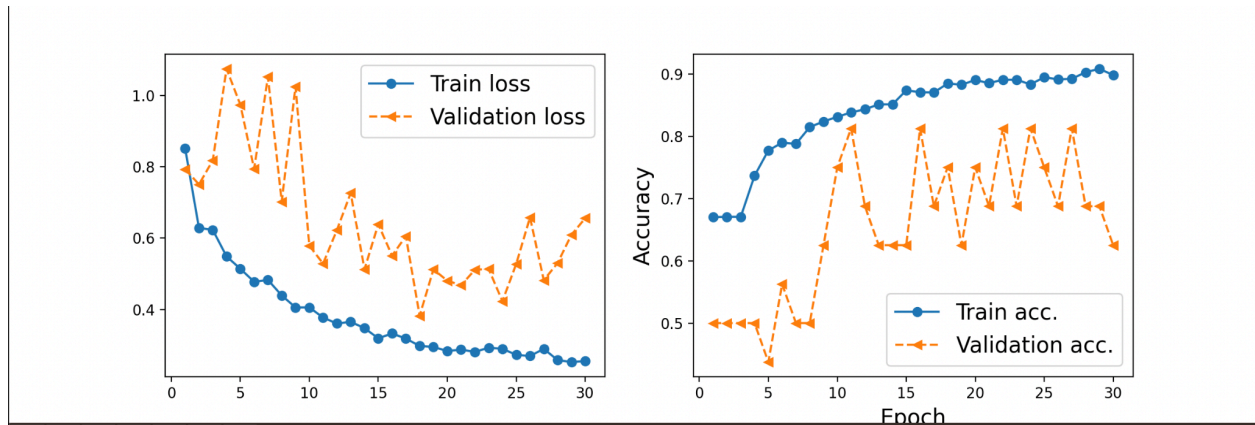


The model ended up performing drastically better on the validation set. I tried again, this time bumping the number up to 0.01.

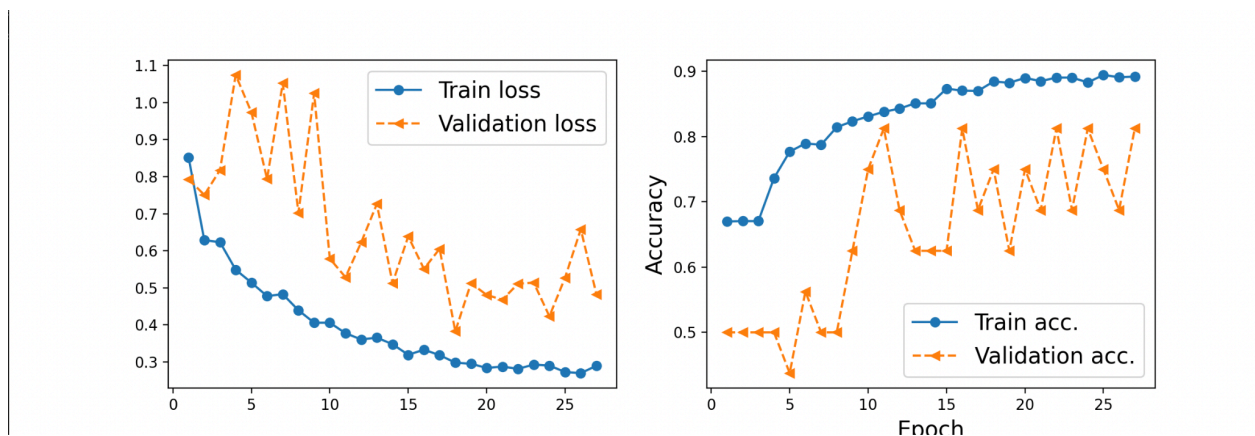


Again, the model performed even better. Next, I decided to change the epochs to 30 because that's where the validation accuracy seems to drop off without much difference in the training

loss at that point. I also removed the dropout layers to get more training samples since it was already low and there's not really a need to use dropout.



The results looked better, however I noticed that it was still trailing off at around 27 epochs. I decided to cut it off there for the best results.



Average validation accuracy: 0.6528
Best validation accuracy: 0.8125

The accuracy is a lot better, however it's not good. For better results, it is probably best to try and augment the data even more to get more samples as the model is overfitting due to a lack of training data.

Support Vector Machine (SVM)

The second algorithm I chose is a Support Vector Machine (SVM). SVMs work by finding the decision boundary that maximizes the margin between the two classes — in this case NORMAL

and PNEUMONIA. Rather than just finding any line that separates the classes, the SVM finds the widest possible street between them, which tends to generalize better to new data. The data points that sit closest to that boundary and actually define it are called support vectors.

I chose SVM as a comparison baseline for the CNN because SVMs are well suited for high-dimensional data, and flattened image pixels are a very high-dimensional feature space. Each image is resized to 32x32 and flattened into a vector of 1,024 pixel values before being passed to the model. While this loses the spatial structure that the CNN preserves, it is a valid and commonly used approach for image classification with traditional ML algorithms. Note that data augmentation techniques like random cropping and flipping were not applied here — unlike the CNN, the SVM has no spatial awareness and would treat a flipped image as a completely unrelated data point rather than a useful variation of the same image.

Transformers

Because SVM requires flat numeric input, the preprocessing pipeline is different from the CNN. Each image is converted to grayscale, resized to 32x32, and flattened into a 1D array of 1,024 pixel values. A StandardScaler is then applied to normalize the pixel values to have zero mean and unit variance. This is important for SVM performance because the algorithm is sensitive to the scale of input features — without scaling, pixels with higher intensity ranges would dominate the margin calculation. The pipeline uses sklearn's Pipeline class with two steps: StandardScaler followed by the SVM classifier. No custom transformer class was needed since the image loading and flattening is handled before the pipeline.

Example 1: Linear SVM (LinearSVC, C=1) — 75.2%

The first model is a basic LinearSVC with C=1 and hinge loss, which finds a straight linear boundary in the 1,024-dimensional pixel feature space. It achieved 75.2% accuracy on the test set. This is a reasonable baseline given that we are working with raw pixel values and no kernel — it confirms there is some linear signal in the data, which makes sense because PNEUMONIA images tend to have higher pixel intensities in the lung regions due to consolidation. However, a single linear boundary is not enough to fully separate the two classes given the complexity of the image data.

```
Train set shape: (1000, 1024)
Test set shape: (500, 1024)
Classes: 0=NORMAL, 1=PNEUMONIA
Linear SVM accuracy: 0.752
```

Example 2: Polynomial Features + LinearSVC (degree=2, C=10) — 77.6%

The second model adds explicit polynomial feature combinations to the pixel vectors before passing them to a LinearSVC. This improved accuracy to 77.6%, a small but meaningful gain over the linear model. Adding polynomial combinations allowed the model to capture some non-linear relationships between pixels that a straight boundary could not. The degree was kept at 2 rather than 3 because a degree-3 expansion of 1,024 features would produce an enormous number of combinations and become very slow or infeasible to train.

```
Train set shape: (1000, 1024)
Test set shape:  (500, 1024)
Classes: 0=NORMAL, 1=PNEUMONIA
Polynomial SVM accuracy: 0.776
```

```
Process finished with exit code 0
```

Example 3: Polynomial Kernel SVM (degree=3, C=5) — 74.2%

The third model uses the polynomial kernel trick via the SVC class, which achieves a similar effect to example 2 but without explicitly computing the expanded feature set. Despite this, accuracy dropped to 74.2% — lower than both previous models. This is likely because degree=3 with C=5 caused the model to overfit to the training data. As the slides note, if the model is overfitting you should reduce the polynomial degree. This result illustrates that a higher degree kernel does not always mean better performance, and careful tuning is required.

```
Train set shape: (1000, 1024)
Test set shape:  (500, 1024)
Classes: 0=NORMAL, 1=PNEUMONIA
Poly kernel SVM accuracy: 0.742
```

```
Process finished with exit code 0
```

Example 4: RBF Kernel SVM (gamma=0.001, C=10) — 79%

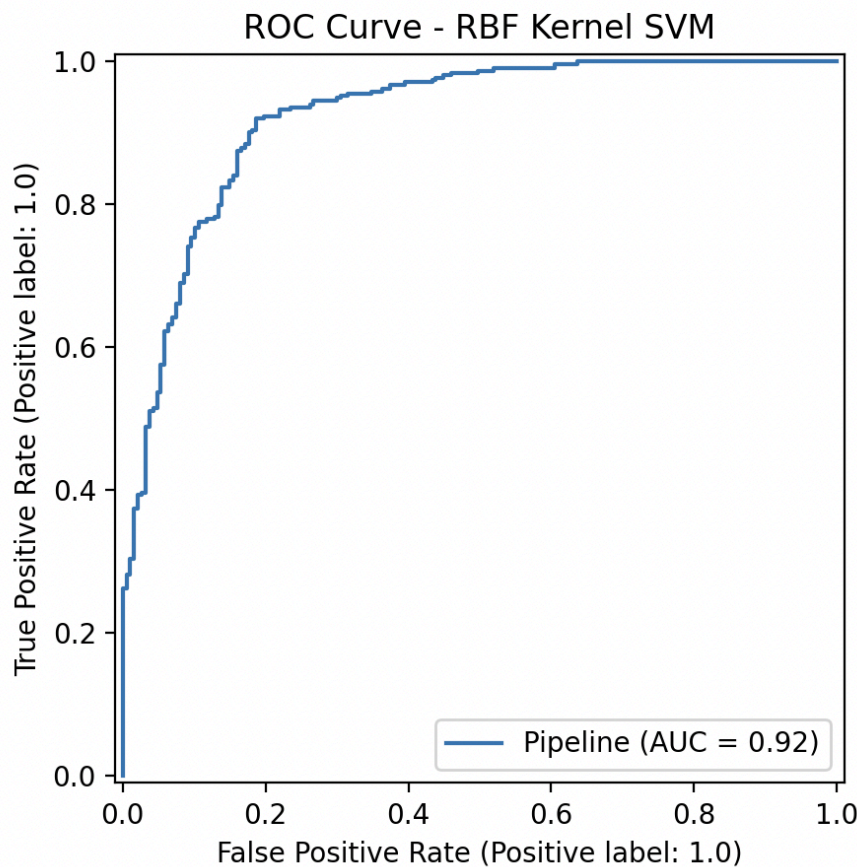
The fourth model uses the RBF (Gaussian) kernel, which is generally the strongest choice for image data because it can model complex non-linear decision boundaries by measuring the similarity between training examples. The initial run with $\gamma=0.1$ and $C=1000$ produced only 62.4% accuracy, worse than the linear model. This is because $\gamma=0.1$ is far too high for a 1,024-dimensional feature space, causing each training point to influence only a very tight local region and leading the model to memorize rather than generalize. After tuning down to $\gamma=0.001$ and $C=10$, accuracy improved to 79%, the best result across all four SVM variants. This confirms the advice from the slides that the RBF kernel works well in most cases, but only when γ and C are appropriately tuned for the data.

```
Train set shape: (1000, 1024)
Test set shape: (500, 1024)
Classes: 0=NORMAL, 1=PNEUMONIA
RBF kernel SVM accuracy: 0.79
```

Summary

Across all four SVM examples, the RBF kernel with $\gamma=0.001$ and $C=10$ performed best at 79%, followed closely by polynomial features at 77.6%, then linear SVM at 76.2%, and polynomial kernel at 74.2%. The results show that the raw pixel feature space does contain enough signal for an SVM to classify chest X-rays above chance, but the spatial structure lost during flattening limits how far any of these models can go. This is reflected in the fact that even the best SVM result is expected to fall short of the CNN, which preserves and learns from the spatial arrangement of pixels directly.

The ROC curve for the best performing model, the RBF kernel SVM, shows an AUC of 0.92, which is a strong result. The curve climbs steeply toward the top-left corner early on, meaning the model correctly identifies most pneumonia cases before accumulating many false positives. This is especially important in a medical context where missing a true pneumonia case is more costly than a false alarm. The gap between the 79% accuracy and the 0.92 AUC is also worth noting, accuracy counts every prediction equally at a fixed threshold of 0.5, while ROC measures how well the model ranks pneumonia cases above normal ones across all possible thresholds. The high AUC suggests the model's probability scores are well-calibrated even when its hard predictions at the 0.5 threshold are not always correct.



Example 1: Decision Tree (Cross-Validation) — 85.6%

```
Train set shape: (1000, 1024)
Classes: 0=NORMAL, 1=PNEUMONIA
DT CV accuracy scores: [0.85 0.87 0.78 0.8 0.89 0.84 0.88 0.88 0.91 0.86]
DT CV accuracy: 0.856 +/- 0.038
```

The Decision Tree model was evaluated using 10-fold cross-validation to obtain a more reliable estimate of its performance. The model achieved an average accuracy of **85.6% ± 3.8%** across the folds. This indicates that the model performs reasonably well on the training data, but the relatively higher standard deviation suggests some variability depending on how the data is split.

Compared to earlier single-split results, cross-validation provides a more stable estimate of generalization performance by training and validating the model on multiple subsets of the data. However, the variation across folds also suggests that the Decision Tree is somewhat sensitive to the specific training examples it sees, which is a known limitation of single decision tree models.

Example 2: Decision Tree Hyperparameter Tuning (RandomizedSearchCV) — 87.2%

```
Train set shape: (1000, 1024)
Classes: 0=NORMAL, 1=PNEUMONIA
Best score: 0.8720000000000001
Best params: {'decisiontreeclassifier__min_samples_split': 5, 'decisiontreeclassifier__min_samples_leaf': 2, 'decisiontreeclassifier__max_depth': None, 'decisiontreeclassifier__
```

To improve the Decision Tree model, RandomizedSearchCV was used to explore different hyperparameter combinations. The best cross-validation score achieved was **87.2%**, which is a slight improvement over the baseline cross-validation result of 85.6%.

The best parameters were:

- `max_depth = None`
- `min_samples_split = 5`
- `min_samples_leaf = 2`
- `criterion = gini`

These results indicate that allowing the tree to grow without a strict depth limit, while controlling the minimum number of samples required to split and form leaf nodes, helps balance model complexity and generalization. In particular, increasing `min_samples_split` and `min_samples_leaf` reduces overfitting by preventing the tree from creating overly specific splits based on small subsets of data.

Overall, hyperparameter tuning provided a modest improvement in performance, but the gains were relatively small. This suggests that while Decision Trees can capture useful patterns in the data, they are still limited in their ability to generalize compared to ensemble methods like Random Forest

Example 3: Random Forest (Cross-Validation) — 93.1%

```
Train set shape: (1000, 1024)
Classes: 0=NORMAL, 1=PNEUMONIA
RF CV accuracy scores: [0.98 0.93 0.96 0.95 0.85 0.91 0.94 0.94 0.91 0.94]
RF CV accuracy: 0.931 +/- 0.034
```

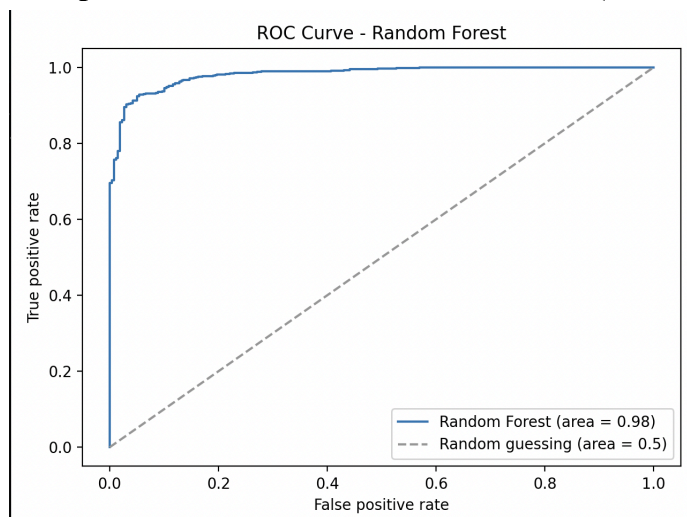
The Random Forest model was evaluated using 10-fold cross-validation and achieved an average accuracy of **93.1% ± 3.4%**, which is a significant improvement over the Decision Tree results.

This demonstrates the effectiveness of ensemble learning in reducing overfitting and improving generalization.

By combining multiple decision trees trained on different subsets of the data, the Random Forest reduces the variance associated with a single tree. This leads to more stable and consistent performance across different folds. The relatively low standard deviation compared to the Decision Tree also indicates that the model is less sensitive to variations in the training data.

These results confirm that Random Forest is better suited for this problem than a single Decision Tree, especially when working with high-dimensional image data. The ensemble approach allows the model to capture more complex patterns while avoiding the instability seen in individual trees.

Example 4: ROC Curve — Random Forest (AUC ≈ 0.98)

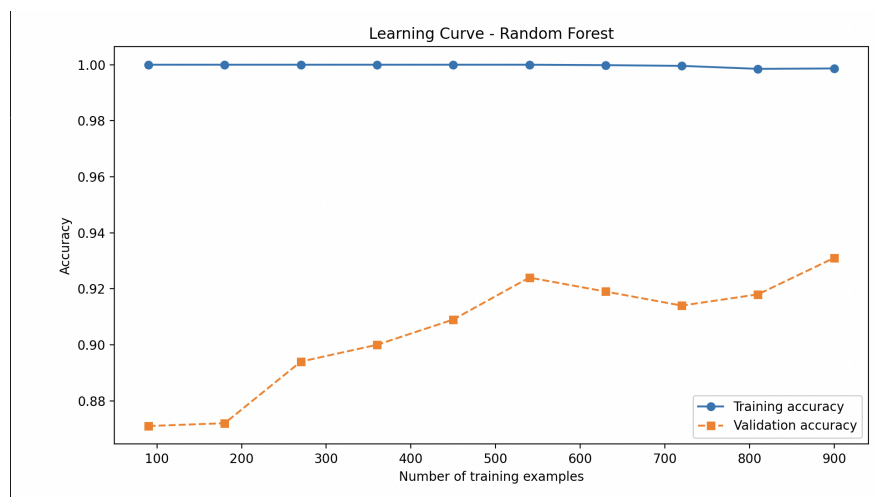


The ROC curve for the Random Forest model shows an AUC of approximately **0.98**, which is an excellent result. The curve rises very steeply toward the top-left corner, indicating that the model is able to correctly identify a high proportion of pneumonia cases while maintaining a very low false positive rate.

Compared to the Decision Tree, this represents a major improvement in classification performance. The high AUC confirms that the Random Forest is not only accurate at a fixed threshold, but also very effective at ranking pneumonia cases above normal cases across all possible thresholds.

Similar to the SVM results, there is a noticeable difference between accuracy and AUC. While cross-validation accuracy measures performance at a fixed decision boundary, AUC evaluates the model's ability to separate classes overall. The very high AUC suggests that the model's predicted probabilities are well-calibrated and that it captures strong underlying patterns in the data.

Example 5: Learning Curve — Random Forest

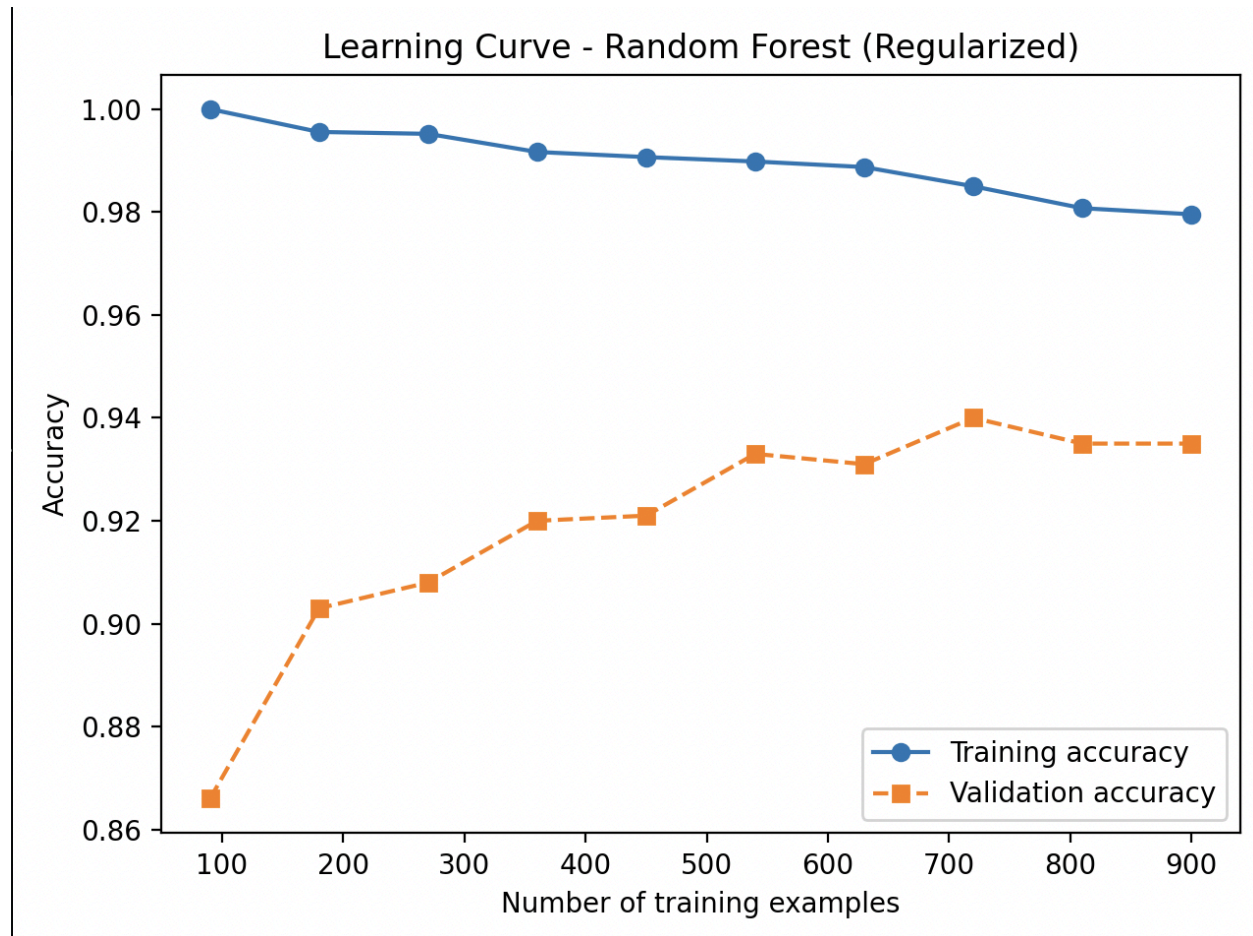


The learning curve for the Random Forest model shows a clear gap between training and validation accuracy. The training accuracy remains extremely high, close to 100%, across all training set sizes, while the validation accuracy starts lower and gradually increases as more data is added, reaching around 93%.

This large gap between training and validation performance indicates that the model is overfitting. The model is learning the training data very well but is not generalizing as effectively to unseen data. This behavior is typical for highly flexible models like Random Forest when the individual trees are allowed to grow too deep.

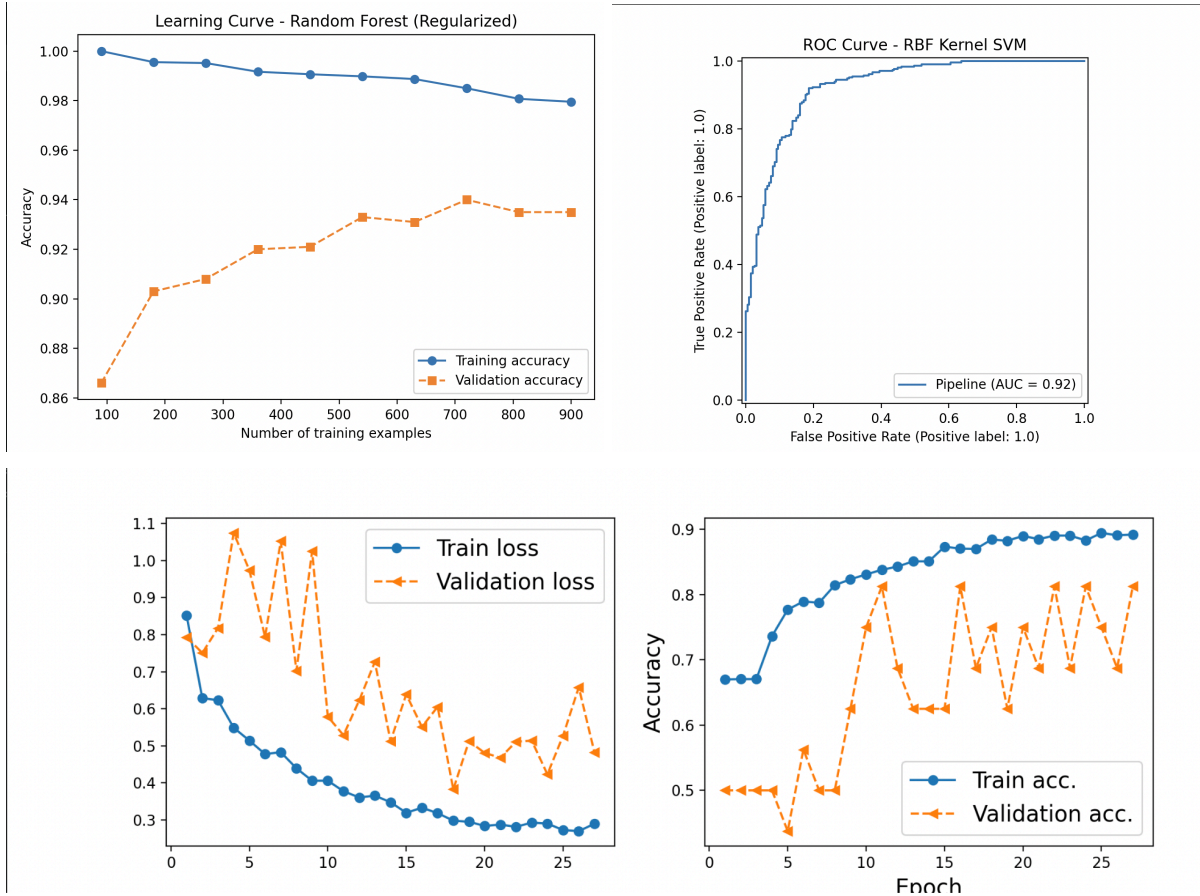
However, the validation curve does show steady improvement as the number of training examples increases, suggesting that adding more data helps reduce overfitting. Despite this, the persistent gap indicates that model complexity still needs to be controlled.

This observation motivates the need for regularization, which will be addressed in the next example by limiting tree depth and adjusting other hyperparameters to improve generalization.



After applying regularization, the learning curve shows a smaller gap between training(98%) and validation(94%) accuracy, indicating that overfitting has been reduced. This is pretty decent, only around a 4 point difference between values. Suggesting that the model had trained well. This is the best across all the models so far.

Final Part 2 Conclusion



In this project, three different machine learning approaches were explored for classifying chest X-ray images as NORMAL or PNEUMONIA: a Convolutional Neural Network (CNN), a Support Vector Machine (SVM), and a Random Forest classifier.

The CNN, while being the most appropriate model for image data, did not perform as well as expected in this stage. It achieved an average validation accuracy of approximately **65%**, with a best validation accuracy of around **81%**. This suggests that the model struggled to generalize effectively, likely due to limited data, insufficient tuning, or training instability.

The SVM performed better, reaching approximately **79% accuracy**, showing that even with flattened pixel data, it can capture meaningful patterns. However, its performance was still limited by the loss of spatial structure in the images.

The Random Forest achieved the best performance among the models, with a cross-validation accuracy of approximately **93%** and an AUC close to **0.98**. This demonstrates the strength of ensemble learning in handling high-dimensional data and reducing overfitting compared to a single Decision Tree.

Overall, the results show that while CNNs are theoretically the best choice for image data, they require careful tuning and sufficient data to outperform traditional machine learning models. In this case, the Random Forest provided the most reliable performance. Future improvements could include better CNN tuning, more data augmentation, and longer training to fully leverage its potential.